Learning What to Change: the poster, explained

Short speaker explainer for the Qwen-architecture review poster, 19 October 2026. REVIEW DRAFT for Toni, Chris and Pierce before anything is printed.

- **How to use it.** Read the plain text aloud. Grey bracketed lines are notes to you.
- **Numbers.** Every number is in `records/CORA_SUMMIT_CLAIM_LEDGER_20261017.md`, with its scope, and is audited against the poster in `POSTER_CLAIM_AUDIT.md`.
- **Figures.** Real figures are drawn from hash-pinned recorded data. Illustrative figures say so.
- **The longer research dossier.** A deeper companion is kept separate; it is not available on Athe for this session. This explainer relies only on the records named here.

The 20-second version

I study how a reasoning system can tell what to change when its explanation of a problem is not good enough: the objects it sees, the rule it uses, or simply the evidence it has. My system, CORA, solves ARC puzzles by writing small programs. It once built a new rule on a real puzzle and checked it carefully. But a version that ignored its failure record did the same, and on 58 more puzzles construction added nothing. Learning which change helps is the open problem I am now testing. The next step, not yet built, is to let a neural model learn which objects and rules to propose, while CORA executes and checks them.

The 90-second version

[full script, with pointing cues: POSTER_90_SECOND_SCRIPT.md]

- **The question.** When a reasoner's explanation is inadequate, can it work out whether it needs different objects, a different rule, or better evidence, and choose a change that improves predictions on new problems?
- **The test.** ARC gives two or three examples and a new input, so it is a clean place to test this.
- **Fitting is not explaining.** On 60 exposed training puzzles, CORA's checker accepted 6 answers, all correct. But on 26 puzzles every rule that fit the examples failed the test.
- **A rule CORA built.** On one real puzzle CORA built a new conditional rule: blue objects send a line up-left, red ones down-right. It rebuilt the rule three times with an example held out, and it was right.
- **The limit.** A version that ignored the failure record found the same rule, and on 58 further puzzles construction added zero verified solves.
- **Next.** CORA-v2 can already notice some inadequacy, such as competing hypotheses or an example the rule rests on alone. What is not yet shown is choosing the revision that helps.
- **The neural lane.** My reproduction of the NVARC method scores 27.64; it is not CORA. The proposed loop: Qwen learns a prior over objects and executable rules, CORA executes and checks them, and feedback helps revise the next hypothesis. It is not built yet.

The two-minute version

Add to the 90-second version:

- **Three ways an explanation fails** (the illustrative panels): wrong objects, a transformation the rule language cannot express, too little evidence. Memorizing is a fourth. A failing system does not know which one happened.
- **What the 0 of 58 suggests.** An evaluator-only check suggests most lie outside the single-rule language construction used: a hypothesis about expressivity, not proof of new semantics.
- **The neural baseline.** My reproduction of the third-party NVARC method scores 27.64; it is not CORA. Native CORA and CORA with rule construction score 0.00.
- **Biology.** What might transfer is the revision policy, not the grid operations: future work, untested.

1. The question: why objects, rules, evidence and revision are different problems

[poster banner]

The question, shaped in conversation with Toni Kazic: can a system induce useful representations and executable rules from a few observations, recognize when its explanation is inadequate, decide what must change, and carry that way of revising into a domain whose meaning is different? These are research aims. CORA has not achieved them all.

Words used on the poster, and here.

- **Representation**: how a grid is broken into objects and parts.
- **Rule**: executable instructions that change the grid.
- **Grammar** or **rule language**: the set of rules a system can express at all.
- **Memorization**: copying apparent regularities in the examples without a relation that transfers.
- **Evidence gap**: the examples do not distinguish between plausible hypotheses.
- **Leave-one-out**: rebuild the rule with one example hidden, and check that it predicts that example.
- **Failure diagnosis**: identifying an *observable* reason to doubt or revise the current explanation. It does not mean knowing that the hidden test answer is wrong; CORA never sees it.
- **Certified**: unique, and rebuilt in every leave-one-out check.
- **Exploratory**: executable and consistent with every example, but not certified. Never counted as a verified solve.
- **Generalization**: correct answers on new task instances, evaluated separately under a declared exposure boundary.

Why the four are different problems. The same failure, a wrong answer, can come from four places, and each needs a different repair:

- the *objects* were wrong: revise the representation;
- the *rule language* lacked the operation: extend the language;
- the examples did not *decide*: keep alternatives, abstain, or ask for evidence;
- the rule was *memorized*: a fit that does not transfer.

The wrong repair can break what already worked. For example, adding rules everywhere can turn a unique answer into a tie (page 7). The research problem is to tell these apart from the examples alone, choose a repair, and show that the choice beats doing everything blindly.

2. Why reasoning fails (poster column 1)

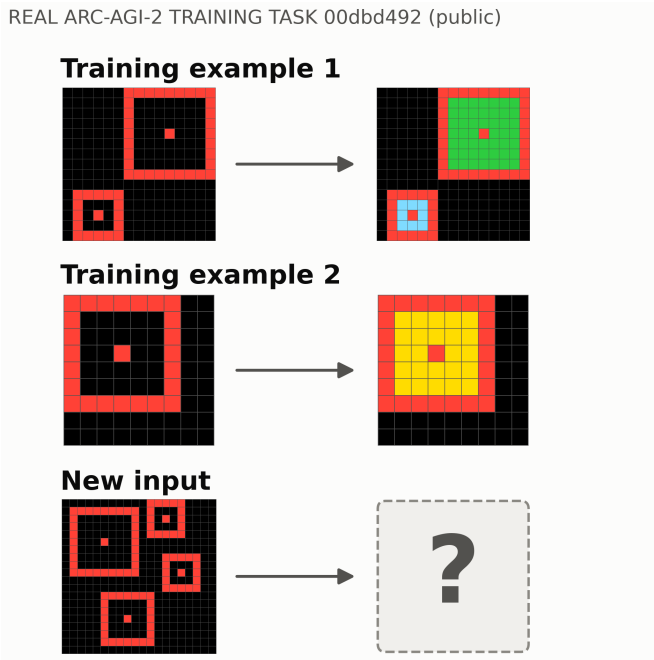
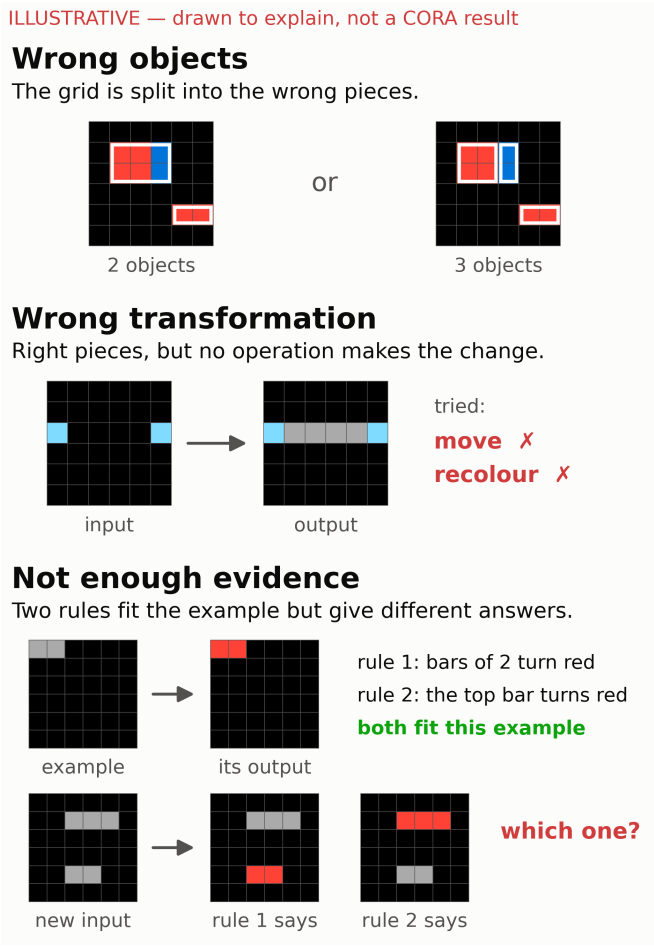


Figure: real task 00dbd492, a public ARC-AGI-2 training task. Grids are drawn cell for cell from the recorded bundle `v2_witness_00dbd492.json` (in `docs/summit_demo_20261023/data/`).

- **Semantics.** A grid is a small array of colours 0–9 (0 is black). A task gives a few training pairs (input → output) and a new input.
- **The task.** The solver must produce the new output exactly, with two attempts allowed. Here each frame is filled with a colour set by its size.
- **Memorization versus generalization.** A rule like "paint the cells that changed in example 1" fits example 1 and fails elsewhere: that is memorization. "Fill each frame with the colour its size calls for" generalizes.
- **Exposure.** This task was examined during CORA-v2 development; it is used here only to explain ARC.

Three ways an explanation fails (the illustrative panels)

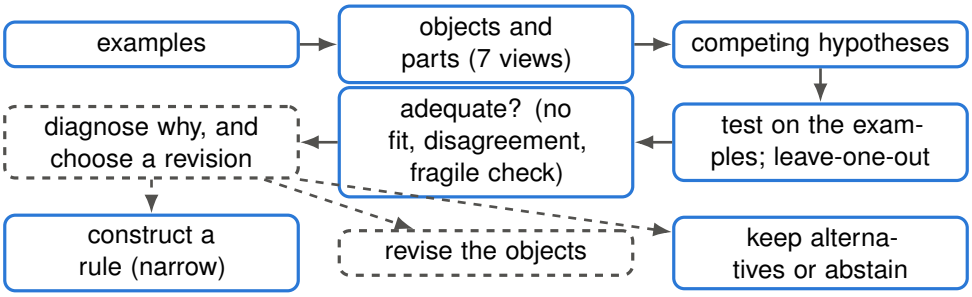


ILLUSTRATIVE: drawn to explain, not an ARC task and not a CORA result.

- **Wrong objects.** The same grid splits into 2 or 3 objects, depending on whether touching cells of different colours count as one object. A rule written for one split can fail on the other.
- **Wrong transformation.** The right objects, but the available operations (move, recolour) cannot draw the connecting line: the rule language lacks the operation.
- **Not enough evidence.** "Bars of 2 turn red" and "the top bar turns red" both fit the example and give different answers on a new input.
- **A fourth: memorization.** A rule that copies the example's output fits it, then fails a new input (the 26 of 60 in section 5).

3. CORA today: what is built, and what is a goal

[poster column 3, the symbolic lane]



Solid outlines: implemented and tested. Dashed: proposed, or not yet validated.

- **Native CORA (v23).** The frozen original system.
  - It segments a grid into objects in seven ways ("views").
  - It searches for an object program that reproduces every example.
  - It accepts only answers that survive leave-one-out rebuilding.
  - Its whole harness adds other solver layers and gives two attempts.
- **CORA-FHR** (failure-conditioned hypothesis revision), the rule-construction experiment.
  - It reads native's own failure trace and constructs a missing growth operation from a typed grammar of 37,750 expressions (G-grow).
  - It installs the operation reversibly and accepts it only if the unchanged engine uses it and every fold rebuilds it.
  - A **witness** is one recorded run that shows the whole chain end to end.
- **CORA-v2**, the hypothesis reasoner.
  - It keeps every hypothesis (representation, geometry, colour policy) that reproduces every example: its **version space**.
  - It abstains when they disagree, and reruns the search with each example held out.
  - It labels why it is unsure, with observable *diagnoses*, and offers exploratory guesses separately from certified answers.
- **Proposed: Neural-CORA** (section 8). One CORA-owned reasoner in which a learned neural prior proposes objects and executable rules, and CORA executes, checks and revises them. It is not built. The only existing piece is a unit-tested CORA-side check of neural candidate grids.
- **What is a goal, not a result.** Choosing the revision from the diagnosis (dashed). Revising the object representation (dashed). Transfer to science (section 11).

4. The real-task rule: ka95aa62f67d9 (poster column 2)

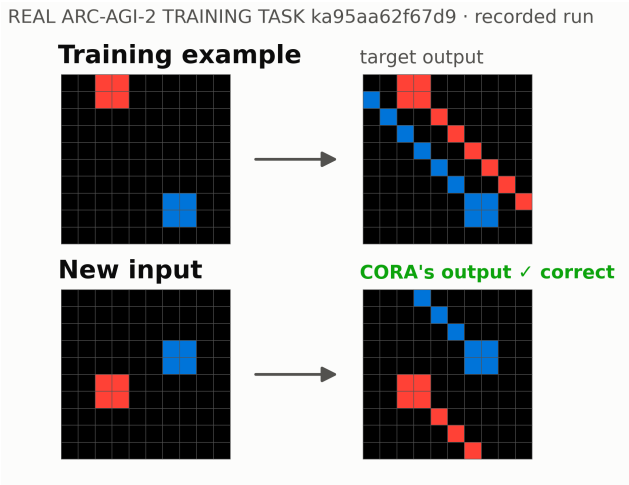


Figure. Real grids from the pinned bundle (`real_task.json` b06073a7, `real_witness.json` f0da3a8d): a training example with its target output, and the new input with CORA's output.

- **The rule CORA constructed.** If an object's colours are {1}, a ray goes north-west from its centre to the border, otherwise south-east. In ARC colours, **blue (1) objects send a line up-left; red (2) objects down-right**, each in the object's own colour.
- **The checks.** The rule was installed reversibly in the unchanged engine and rebuilt in 3 of 3 leave-one-out folds. The answer equals the evaluator's answer.
- **The comparison.** Frozen native CORA and matched extra search do not solve this task.

Why this is not hidden-set generalization.

- The failure autopsy chose this task, in-sample, and it is an exposed public training task.
- A failure-blind constructor, given the same evidence, found the same rule. So the run shows that CORA can *construct and verify* a missing rule. It does not show that diagnosing the failure *caused* the gain (a **causal** claim needs a controlled comparison that the diagnosis wins).
- It was developed after the CORA-FHR Kaggle entry and is not part of it.

The offline demo. `sh docs/summit_demo_20261023/run_demo.sh -real` needs Python with numpy. It is a *recorded replay*:

- it prints the recorded task, native's failure, the constructed rule, the engine check and the evaluator's verdict;
- it re-executes the constructed operation as standalone code (`guard_standalone.py`) on the test input's objects.

It is not a live rerun of the full engine. It was validated on 19 October (exit code 0).

5. What the experiments taught: the 60-task audit and the 58-task negative

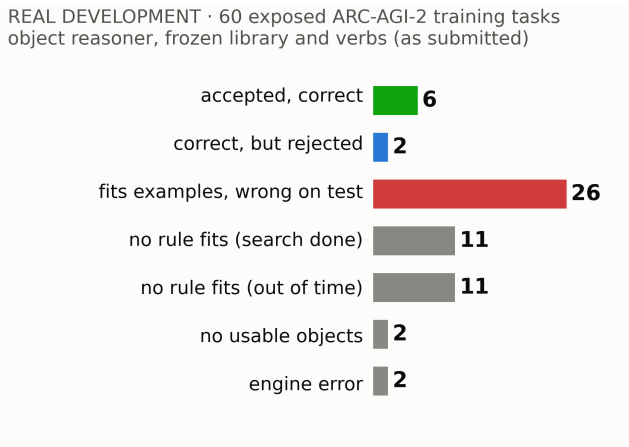
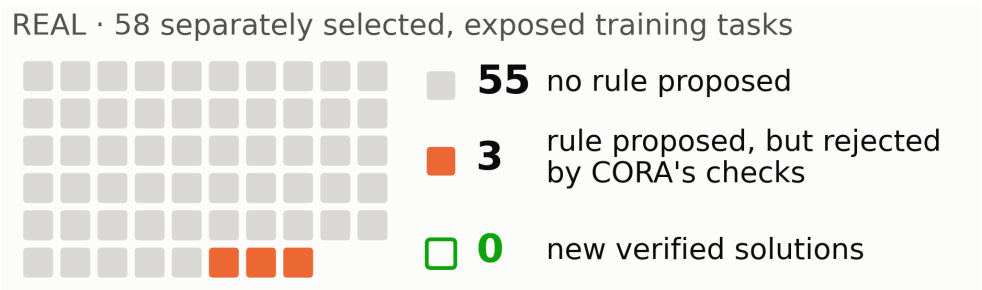


Figure: the 60-task audit. Real data (`eval_dev60_native_assets.json`, 54d8d744). It covers the native object reasoner with its frozen library and learned verbs, as submitted, on 60 exposed ARC-AGI-2 training tasks.

- Accepted and correct: 6. Accepted and wrong: 0.
- 26 tasks had rules that fit every example and were all wrong on the test: **memorization**.
- A correct rule existed but was rejected by the check on 2 tasks; no rule fit on 22.
- This is one branch. The whole harness got 12 of 60 over two attempts, and the two are never mixed.

The 58-task generalization test (negative)



- **Design.** 58 separately selected, still exposed training tasks; a protocol frozen before the run. Arms: frozen native, matched extra search, construction from the failure record, failure-blind construction, shuffled evidence, and a contrast arm.
- **Result.** No construction arm produced a certified answer (0 of 58), with 0 wrong and 0 lost. Frozen native solved 1. There were no discordant pairs, so the failure record showed no advantage, and the version was stopped.
- **The figure** (poster column 2), one square per task, for construction from the failure record (arm EG; `outputs/cora_fhr/rows_standin58_gen.jsonl`): no rule was proposed on 55; on 3 a rule was proposed but rejected (2 by the unchanged engine, 1 by the leave-one-out rebuild); 0 verified. The failure-blind arm: 52, 5 and 1, also 0.

Why so few constructions fire: a restricted diagnostic, not a proof

An evaluator-only (**oracle**) analysis, which reads the answers and is used only for diagnosis, asked one narrow question. Can a *single, unguarded, uniformly applied* G-grow expression explain the changed cells in any of native's seven views?

- For 50 of 58 tasks: no. In the sense defined here, they are **inexpressible** in that class.
- Of the other 8, 5 need colour behaviour no single such expression provides, and 3 pass the necessary checks.

This does *not* show that the tasks are impossible for CORA or need entirely new semantics. Guarded, deeper or re-segmented programs were not tested. It supports an expressivity hypothesis. Never say "56/58 impossible".

6. CORA-v2: many hypotheses, ambiguity and fragile checks (poster column 3)

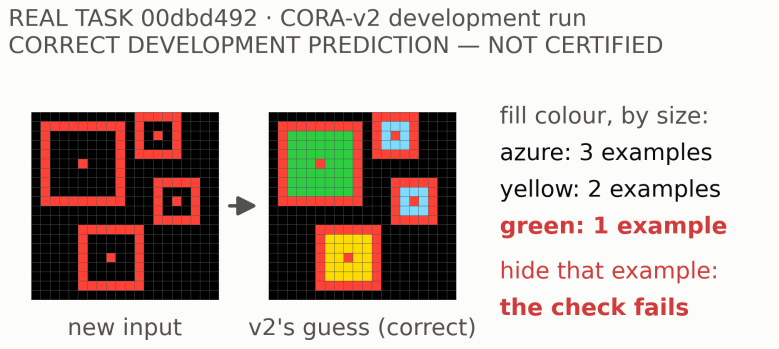


Figure: task 00dbd492, real data from the v2 engineering run.

- **The guess.** v2's 1,365 consistent hypotheses all agree, and the guess is correct. Frozen native and matched search have no certified answer.
- **Why it is not certified.** The test needs the colour for the size-32 frame (green), and only one example shows it. With that example hidden, the rebuilt rule has no entry for size 32 and abstains: it does not predict wrongly.
- **The diagnosis.** Checks on the examples rule out ambiguity, a wrong representation and a defect. The cause is *insufficient evidence*. A failure-blind reasoner gives the same answer.

It is an **exploratory**, uncertified prediction on an exposed task.

Engineering evidence on 118 exposed development tasks (in-sample)

	growth only (frozen v2)	growth + object selection
certified	0	0
58 pool tasks: exploratory correct / wrong	3 / 1	4 / 0
60 dev tasks: exploratory correct / wrong	1 / 0	2 / 0
mean CPU per task	20 s and 12 s	40 s and 27 s

- **More rules can mean less certainty.** On 22eb0ac0, adding object selection everywhere turned a unique answer into a disagreement. That is a concrete reason to revise where a diagnosis asks for it, not everywhere.
- **Where whole-object operations fall short.** On 22 development tasks objects change only in part:
  - native's built-in object parts fully explain 2 and partly explain 8;
  - 7 are unexplained, often by parts that touch other objects;
  - 5 show no partly changed object.

  A two-stage whole-object transformation produced 0 predictions on 32 routed tasks. These are planning results, not accuracy.
- **Two attempts** (evaluator-side replay, no submission). Adding v2's exploratory guesses to native's recorded attempts gives:
  - 58 pool tasks: 7 $\rightarrow$ 8 when filling only duplicate second attempts, 9 when always the second attempt;
  - 60 dev tasks: 12 $\rightarrow$ 12;
  - no answers lost.

  This is in-sample, so not evidence for an entry.

7. Inventing new operations: the automatic rule-language extension (Step B)

[moved here from the poster; the poster's footer points to this section]

What it is. Step B is a separate CORA Level-4 experiment, in its own repository and run. It asks whether a system can construct genuinely new, reusable operations instead of depending only on operations a researcher specified.

- **K2.** It uses a frozen, generic, typed constructor language: a finite, closed set of type-directed semantic constructors, in 7 families of 19 schemas.
- **The run.** K2 is applied uniformly to failure clusters identified earlier, in a single run of 62 clusters.
- **What it must separate.** Repairing an existing learner versus inventing a new **semantic extension**: an operation whose meaning the old language could not express.

Status, 19 October 2026, from its counts-only administrative watcher.

- 475 of 497 K2 proposal units are done, with no errors.
- No completion marker and no output hash yet.
- No verdict on novelty or held-out transfer.

What still has to happen. When proposal generation finishes, the outputs are frozen and hashed before anyone inspects them. Then come a bounded novelty check and the reserved transfer evaluation. Finishing the proposals is not evidence of invention.

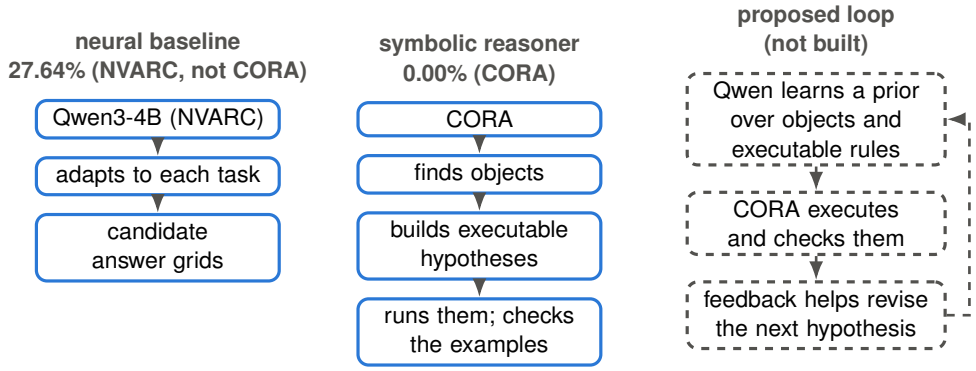
How it relates to CORA-FHR and CORA-v2. All three ask what to do when the current language is not enough.

- FHR constructs a missing operation from a fixed growth grammar for one task.
- v2 compares hypotheses and diagnoses why they are not enough.
- Step B tests whether a generic constructor language can produce new reusable operations.

Step B's code is **not integrated** into CORA-v2 or any Kaggle entry, and none of its protected outputs were read for this poster.

8. From a neural baseline to executable hypothesis revision (poster column 3)

[poster column 3: the two solid lanes and the small dashed loop]



Why I reproduced NVARC, and what I changed. I reproduced it to learn what a strong learned system achieves under the same Kaggle conditions, and what CORA lacks by comparison. NVARC is the NVARC team's published ARC Prize 2025 method. I deployed it from their public notebook (`records/KAGGLE_NEURAL_V1_ORIGINALITY_AUDIT_20261014.md`), with six changes, all for deployment: compatibility, coverage, diagnostics, provenance and robustness. Unchanged: the Qwen3-4B checkpoint, which the NVARC team trained on their synthetic data; their per-task test-time training; their decoding; their scoring and selection.

What test-time adaptation contributes. For each task, NVARC briefly trains a small adapter (LoRA, rank 256) on that task's demonstrations, with augmentations. It then decodes candidate grids and scores them. This is learning *within* a task. It is not evidence of reusable learning across tasks, and a model adapted on the demonstrations is not an independent check of its own answers.

Why 27.64% belongs to NVARC. The method, model and adaptation are theirs, and I changed none of them. The score is the official public score of my deployment of their method (Kaggle 57160429). It is not a CORA result, and it would not be a Neural-CORA result.

Why combining answer grids added nothing
(`records/NVARC_CORA_COMPLEMENTARITY_AUDIT_20261014.md`).

- On 240 exposed stand-in tasks (mechanics only, since both systems had seen them), NVARC got 248 of 259 outputs and CORA 47. CORA solved none that NVARC missed.
- Filling NVARC's 147 empty second attempts with CORA's answers gained 0 and lost 1: a 1×1 answer that NVARC's placeholder had matched by coincidence.
- On the hidden set CORA scored 0.00, so its grids could add nothing there.

The open question is whether neural *proposals* can help CORA *find* hypotheses it now misses.

The proposed single architecture: Neural-CORA

It is one inference algorithm, owned by CORA. It is not a router, an ensemble or a second-attempt filler.

- **CORA owns** the typed hypothesis state (objects, parts, relations, consistent programs, uncertainty, cost), the interpreter, verification, revision actions, budgets and the final two attempts.
- **Qwen contributes inside CORA**: learned scores over representations and typed programs, part or correspondence proposals, candidate ranking and, later, a policy over revisions.
- **The loop**: propose $\rightarrow$ type-check $\rightarrow$ execute on the demonstrations $\rightarrow$ record an observable trace (wrong cells, unsupported conditions, conflicts, fragile checks) $\rightarrow$ choose a revision (representation, program, guard, colour rule, budget, keep alternatives, abstain) $\rightarrow$ repeat within a budget $\rightarrow$ rank the program-grounded predictions.
- **Ruled out**: switching to NVARC's answer when it is confident; NVARC's first attempt plus a CORA

second; voting between notebooks; CORA rationalizing a Qwen grid after the fact.

How it would learn. Today's checkpoint emits grids, not programs. So a small new adapter is trained, for a proposal or ranking head.

- **Targets come from execution.** Proposals that reproduce every demonstration are positives; well-typed failures are hard negatives. Programs with identical results are grouped. Unsupported operations are rejected.
- **A surrogate loss**: program-token likelihood, pairwise ranking or reward-weighted imitation, because exact grid match is not differentiable. It is checked to track executable correctness.
- **Revision training** comes from controlled counterfactual changes, never from hidden answers. If free decoding fails, the first step is to rank safely enumerated CORA programs.
- **Data**: generated, execution-verified task–program pairs. Whether NVARC's own generators may be used is being audited for licence, format and contamination.

How CORA would test a hypothesis.

- A fixed interpreter runs typed programs: static syntax trees, never generated Python.
- A hypothesis is accepted only if it reproduces every demonstration.
- It is certified only if it is rebuilt with each demonstration held out, under the unchanged checks.
- Proposals from a prior adapted on all the demonstrations are labelled *neural-assisted, not independently certified*, unless the adaptation is repeated for each fold.

What would show genuine complementarity. Every arm uses the same inputs, with compute accounted.

arm	what it controls
N0, N+	unchanged NVARC; NVARC with matched extra compute
C0, C+	frozen CORA-v2; CORA-v2 with matched extra search
L0	an untrained or shuffled prior: does training matter?
L1, L2, L3	the trained prior; plus executable feedback; plus a learned revision choice
L3-blind, L3-shuffled	no failure trace, or a shuffled one: does the diagnosis matter?
L3-no-program	program inputs removed: is the symbolic part decorative?

A gain must be new correct predictions on held-out task families and on independently chosen real tasks. It must not be explained by extra compute (N+, C+), an untrained prior (L0) or bypassing the program (L3-no-program).

Three ways the systems could merge. This compares designs, not results.

design	strength	main risk
NVARC-first: Qwen learns to propose, run and revise programs	starts from a strong solver	needs program supervision; NVARC's programs are Python, not typed CORA operations; licence; contamination
CORA-first (Neural-CORA): CORA learns representations, search priorities and revisions	inspectable; closest to the research question	CORA's language limits what any prior can propose (the 0 of 58)
Unified: one learned hypothesis state (representation, program, guard, uncertainty)	the long-term goal	largest effort; still needs exact execution and independent tests

The first experiment is CORA-first, by the user's directive. The shared parts serve all three: the typed language, the interpreter, the execution-verified data and the evaluation harness.

Before claiming generalizable reasoning, all of these are needed:

- a preregistered pilot;
- held-out synthetic task families, then an independently chosen real cohort with a contamination audit;
- paired gains against C+ and N+, with harms counted, fixed seeds and reported uncertainty;
- ablations showing that the learned weights and the executed programs cause the gain;
- for a competition entry, a larger authorized validation and the user's approval of the exact version.

A higher score alone would not show that the revision mechanism caused it.

Limitations.

- NVARC already scores 27.64%, and a program-only reasoner is much less expressive today, so matching it quickly is not expected.
- A neural prior cannot propose what the interpreter cannot express.
- Contamination and fold leakage apply.
- NVARC's method and score remain theirs.

Status, 19 October.

- The architecture is proposed. It is not implemented or validated.
- The only existing piece is CORA-side code that checks neural candidate grids (`cora_neural_fusion/`). It is unit-tested on synthetic tasks only and has never run on real NVARC outputs.
- Started: a read-only audit of both architectures, NVARC's synthetic programs and the licences.
- Next: the typed hypothesis language, then a bounded, preregistered pilot.

9. Where this sits in prior work

The general loop of failing, revising and checking is not new. The possible difference is narrow and testable.

- **Popper** (Cropper and Morel, *Learning programs by learning from failures*, Machine Learning, 2021). Inductive logic programming that turns failed hypotheses into constraints that prune the search. It overlaps with using failure to guide search. CORA additionally asks which *component* (objects, rule language, evidence) to revise, in grid reasoning.
- **DreamCoder** (Ellis et al., PLDI 2021). Wake–sleep library learning that grows reusable abstractions and a neural search guide. It overlaps with growing a rule language, which is closest to Step B.
- **LILO** (Grand et al., ICLR 2024). LLM-guided program synthesis with library compression and auto-documentation. It overlaps with LLM-proposed programs and abstractions.
- **Model Discovery Agent** (Murphy, 2026). LLM-assisted Bayesian experiment design for data-efficient discovery of mechanistic models: recognizing model inadequacy, proposing new models, choosing experiments. It is closest to the scientific framing.

The narrow claim to be tested. A diagnosis-driven choice of *which component to revise* beats, under matched compute and matched evidence:

- a strong failure-blind choice;
- applying every revision;
- a shuffled diagnosis.

It must also add correct predictions on tasks the system was not developed on. None of these systems is said to be unable to do this.

10. How to test whether diagnosis helps

What a convincing test needs.

- **A fresh cohort.** Tasks not used in development, chosen by a rule declared beforehand.
- **Strong controls.**
  - frozen native;
  - matched extra search;
  - a strong failure-blind revision;
  - every revision applied everywhere;
  - the same at matched compute;
  - a shuffled diagnosis.
- **Harm counted.** Wrong certified answers and lost answers are counted, not only gains.
- **Separate tiers.** Certified and exploratory answers are reported separately.
- **A blind evaluator** that reads answers only after every prediction is frozen.
- **A pre-registered decision rule**, applied without retuning on the evaluated tasks.

A **prospective** test is one whose design and cohort are fixed before any result is seen.

Phase 2a status: RUNNING, RESULT PENDING.

- **Cohort.** 40 public training tasks.
  - They were selected by a declared rule and are independent of all v2 development.
  - A single authorized, metadata-only check excluded Step B's protected splits: 44 of 100 candidates were eligible, and the first 40 were used.
  - They are reused public tasks, so a result is a development evaluation, not a blind prospective test or an unseen-task score.
- **Arms.** Frozen native (A), matched extra search (B), and the frozen growth-only v2 (V).
- **GO rule.**
  - complete coverage and valid hashes;
  - zero wrong certified v2 answers;
  - at least 2 tasks where v2 is right and A and B are not.
- **Reporting.** The result will be added here and to the ledger only after its integrity checks.

If GO, Phase 2b. It would test the causal question: does the diagnosis choose better revisions than applying everything, at matched compute, or a shuffled diagnosis? It needs a new eligible cohort and approval.

If NO-GO. The version is closed. The next step would be one small, evidence-based change: part-aware or relational object parts, chosen from the 22-task characterization on page 7.

11. Science next, and how to reproduce everything

Plausible next domains. FUTURE, NOT TESTED: no biological transfer or mechanism is claimed. What might transfer is the diagnosis-and-revision policy. The operations, evidence and validators must be specific to each domain.

- **Protein conformational ensembles.** Is a structural hypothesis wrong, or is the ensemble representation too coarse?
- **Single-cell and other omics.** Is an effect a shift in average, a change in spread, or a change in the mixture of cell states? The level at which you infer (cells or donors) matters.
- **Maize lesion-mimic phenotyping.** Image geometry and topology tell you where and how lesions form, not which pathway is involved. Inferring a candidate pathway would need independent molecular or perturbation evidence.

Examples, data and wording are subject to Pierce's and Chris's input.

Reproducing what you see

- **Poster figures.** `python scripts/summit_visual_figures.py` writes `figures/*.png` and `POSTER_FIGURE_PROVENANCE.json`. It stops if any pinned input's sha256 differs.
- **Poster PDF.** From `docs/summit_20261023_reframe/`, run `SOURCE_DATE_EPOCH=1791763200 FORCE_SOURCE_DATE=1 pdflatex poster_qwen_architecture_review.tex` (the visual version: `poster_visual_final_review.tex`).
- **Editable PowerPoint.** `python scripts/poster_visual_to_pptx.py -pdf docs/summit_20261023_reframe/poster_qwen_architecture_review.pdf -out docs/summit_20261023_reframe/poster_qwen_architecture_review.pptx`. Diagram labels are inside pictures; edit them in the `.tex`. Changes and the claim audit: `POSTER_QWEN_ARCHITECTURE_NOTES.md`.
- **Demos** (offline; Python with numpy), both validated 19 October:
  - `sh docs/summit_demo_20261023/run_demo.sh` replays the synthetic witness;
  - adding `-real` replays the real ka95aa62f67d9 witness.

  Each replays the recorded run and re-executes the constructed operation standalone.
- **Records.**
  - ledger: `records/CORA_SUMMIT_CLAIM_LEDGER_20261017.md`;
  - v2: `records/CORA_REASONING_V2_*.md`;
  - rule construction: `records/CORA_FHR_*.md`.

12. Terms kept off the poster

The poster avoids these abbreviations and technical words. Here is what each one means in this explainer.

- **LOO (leave-one-out)**: rebuild a rule with one demonstration hidden, then check that it predicts the hidden one. A **fold** is one such rebuild.
- **Certified**: unique, and rebuilt in every leave-one-out fold under the unchanged checks. **Exploratory**: it runs and fits every demonstration but is not certified. It is never counted as a verified solve.
- **FHR (failure-conditioned hypothesis revision)**: CORA-FHR reads native CORA's failure trace and constructs a missing operation from a fixed typed grammar.
- **Grammar** or **rule language**: the set of operations a system can express at all.
- **K2**: Step B's frozen, generic, typed constructor language (7 families, 19 schemas), see section 7.
- **Semantic self-extension**: the system adds an operation whose meaning its old language could not express. This is the question Step B asks. It is not a result.
- **OOD (out of distribution)**: tasks unlike those used in development or training.
- **Oracle (evaluator-only) analysis**: an analysis that reads the answers. It is used only for diagnosis, after predictions are frozen, and never feeds a solver.
- **Causal claim**: a claim that a component caused a gain. It needs a controlled comparison that isolates that component, for example beating failure-blind and shuffled controls at matched compute.
- **Exposed task**: a task already examined during development. Results on it are in-sample.
- **Test-time training (TTT)** and **LoRA**: briefly training a small low-rank adapter on one task's demonstrations before predicting. NVARC does this, with augmentations.
- **Matched compute**: a control that is given the same CPU or GPU budget, so extra search or extra training time cannot be mistaken for a better method.

Credits and disclosure. The research question was shaped in discussions with Toni Kazic. Authorship and credits are pending review by Toni, Chris and Pierce. AI coding assistance was used for parts of the implementation and documentation; the research, decisions and claims are the author's.